

PRODUCT OWNER DOCUMENTATION

Automated Cryptocurrency Price Monitoring and Email Notification System

Software Engineering Project • Scrum Model

Role: Product Owner (Member 1)

Due date: November 1, 2026 at 11:59 PM

1. Product Vision

To provide administrators with a reliable automated system that monitors cryptocurrency prices and sends email notifications when configured price conditions are met, reducing the need for manual price checking.

2. Product Description

The product is an automated cryptocurrency price monitoring system. It periodically checks CoinLore through its free REST API, compares current coin prices with configured minimum and maximum ranges, and sends an email to configured recipients when a condition is met.

Technical direction from the project source: Node.js, PostgreSQL hosted on Neon, Sequelize ORM, CoinLore REST API, Gmail SMTP, Zod validation, scripts for coin/email CRUD, and GitHub Actions for scheduled execution.

3. Product Objectives

- Allow administrators to manage monitored cryptocurrencies.
- Allow administrators to configure minimum and maximum price ranges.
- Allow administrators to manage email recipients.
- Retrieve current cryptocurrency prices automatically.
- Compare current prices with configured ranges.
- Send email notifications when conditions are met.
- Run the monitoring process automatically through GitHub Actions.
- Protect sensitive credentials and handle API, database, and email errors safely.
- Deliver a tested, documented, working increment at the end of each Sprint.

4. Product Owner Responsibilities

- Define and communicate the product vision and objectives.
- Create, prioritize, and maintain the Product Backlog.
- Write and clarify user stories and acceptance criteria.
- Participate in Sprint Planning and explain the highest-priority work.
- Answer requirement questions from developers during the Sprint.
- Review completed stories against their acceptance criteria.
- Accept or reject completed work during Sprint Review.
- Collect feedback and continuously re-prioritize the Product Backlog.

- Maintain Product Owner documentation and decision records.

5. Team Structure

Role	Member	Responsibilities
Product Owner	Member 1	Product vision, Product Backlog, priorities, user stories, acceptance criteria, Sprint Planning input, Sprint Review acceptance, stakeholder/product decisions.
Scrum Master	Member 2	Facilitates Scrum events, helps remove impediments, manages Sprint timing, supports the team and Scrum process.
Developer 1	Member 3	System modeling, development, requirements/testing documentation and implementation.
Developer 2	Member 4	Development, database/API/email implementation and testing.
Developer 3	Member 5	Development, automation/CI, testing, documentation and technical support.

6. Functional Requirements

ID	Requirement
FR-01	The system shall store configured cryptocurrencies.
FR-02	The system shall store minimum and maximum price ranges for each coin.
FR-03	The system shall store email recipients.
FR-04	The system shall retrieve coin prices from CoinLore.
FR-05	The system shall compare current prices with configured price ranges.
FR-06	The system shall send email notifications when a configured range is met.
FR-07	Administrators shall be able to create, read, update, and delete coins using scripts.
FR-08	Administrators shall be able to create, read, update, and delete email recipients using scripts.
FR-09	The system shall run automatically at a configured interval through GitHub Actions.

7. Non-Functional Requirements

ID	Requirement
NFR-01	The application shall use Node.js.
NFR-02	The database shall use PostgreSQL hosted on Neon.
NFR-03	Sequelize shall be used as the ORM.
NFR-04	Zod shall be used for input validation.
NFR-05	Coin prices shall be retrieved through the CoinLore REST API.
NFR-06	Gmail SMTP shall be used for email delivery.

NFR-07	Sensitive credentials shall be stored as GitHub Secrets and not committed to the repository.
NFR-08	The application shall handle API, database, and email errors without crashing unexpectedly.

8. User Stories and Acceptance Criteria

US-01 — Must Have

As an administrator, I want to add a cryptocurrency so that I can monitor its price.

Area: Coin management

Acceptance criteria: The coin is validated and stored successfully; invalid input is rejected.

US-02 — Must Have

As an administrator, I want to view the monitored cryptocurrencies so that I know which coins are being tracked.

Area: Coin management

Acceptance criteria: The system lists configured coins and handles an empty list.

US-03 — Must Have

As an administrator, I want to update a monitored cryptocurrency so that I can correct or change its configuration.

Area: Coin management

Acceptance criteria: An existing coin can be updated and the change is persisted.

US-04 — Should Have

As an administrator, I want to delete a cryptocurrency so that I can stop monitoring it.

Area: Coin management

Acceptance criteria: An existing coin can be deleted and no longer appears in the list.

US-05 — Must Have

As an administrator, I want to configure a minimum price so that the system can identify when a coin reaches my desired range.

Area: Price configuration

Acceptance criteria: A valid minimum price is stored and can be updated.

US-06 — Must Have

As an administrator, I want to configure a maximum price so that the system can identify when a coin reaches my desired range.

Area: Price configuration

Acceptance criteria: A valid maximum price is stored and can be updated.

US-07 — Must Have

As an administrator, I want to add email recipients so that notifications can be delivered to the correct people.

Area: Email management

Acceptance criteria: A valid email is accepted; invalid email input is rejected.

US-08 — Should Have

As an administrator, I want to view configured email recipients so that I can verify who receives notifications.

Area: Email management

Acceptance criteria: Stored recipients are listed correctly.

US-09 — Should Have

As an administrator, I want to update an email recipient so that notification delivery information remains correct.

Area: Email management

Acceptance criteria: An existing recipient can be updated and invalid email input is rejected.

US-10 — Could Have

As an administrator, I want to delete an email recipient so that unwanted recipients no longer receive notifications.

Area: Email management

Acceptance criteria: An existing recipient can be deleted successfully.

US-11 — Must Have

As an administrator, I want the system to retrieve the current cryptocurrency price so that monitoring uses current market data.

Area: Monitoring

Acceptance criteria: The system retrieves a valid price from CoinLore and handles API errors.

US-12 — Must Have

As an administrator, I want the system to compare the current price with my configured range so that it knows when to trigger a notification.

Area: Monitoring

Acceptance criteria: The current price is compared with the configured range and a clear match/non-match result is produced.

US-13 — Must Have

As an administrator, I want to receive an email when the configured price condition is met so that I do not need to manually check the market.

Area: Notification

Acceptance criteria: An email is generated and sent to configured recipients when the condition is met.

US-14 — Must Have

As an administrator, I want the system to avoid sending an email when the configured condition is not met so that I do not receive unnecessary notifications.

Area: Notification

Acceptance criteria: No email is sent when the configured condition is not satisfied.

US-15 — Must Have

As an administrator, I want the system to monitor prices automatically so that I do not have to manually execute the monitoring script.

Area: Automation

Acceptance criteria: The monitoring worker runs automatically and completes successfully.

US-16 — Must Have

As an administrator, I want GitHub Actions to execute the monitoring process on a schedule so that price monitoring happens automatically.

Area: Automation

Acceptance criteria: A workflow exists and executes the monitoring script according to its schedule.

US-17 — Must Have

As an administrator, I want invalid coin and email information to be rejected so that incorrect data is not stored.

Area: Validation

Acceptance criteria: Invalid input is detected, rejected, and not persisted.

US-18 — Must Have

As an administrator, I want sensitive credentials to be protected so that database and email credentials are not exposed publicly.

Area: Security

Acceptance criteria: Secrets are not committed to source code and are supplied through GitHub Secrets/environment configuration.

US-19 — Must Have

As an administrator, I want API, database, and email errors to be handled so that the monitoring system does not unexpectedly crash.

Area: Reliability

Acceptance criteria: Relevant errors are handled and useful error information is logged.

US-20 — Must Have

As an administrator, I want the complete monitoring workflow to be tested so that I can trust the system to monitor prices and send notifications correctly.

Area: Testing

Acceptance criteria: Coin configuration, price retrieval, range checking, email notification, and scheduled execution are tested and documented.

9. Product Backlog

The Product Backlog contains the technical work required to deliver the product. Priority is controlled by the Product Owner and may be refined as new information appears.

Priority	Backlog Item	Area	Related Stories	Priority Class
1	Set up Node.js project	Foundation	US-20	Must Have
2	Set up Neon PostgreSQL database	Foundation	US-01–US-10	Must Have
3	Configure Sequelize	Foundation	US-01–US-10	Must Have
4	Create Coin model	Data model	US-01–US-06	Must Have
5	Create Email model	Data model	US-07–US-10	Must Have
6	Create Coin CRUD scripts	Administration	US-01–US-04	Must Have
7	Create Email CRUD scripts	Administration	US-07–US-10	Must Have
8	Add Zod validation for Coin scripts	Validation	US-01–US-06, US-17	Must Have
9	Add Zod validation for Email scripts	Validation	US-07–US-10, US-17	Must Have
10	Integrate CoinLore REST API	Monitoring	US-11	Must Have
11	Retrieve coin prices	Monitoring	US-11	Must Have
12	Implement price-range checking	Monitoring	US-05, US-06, US-12	Must Have
13	Integrate Gmail SMTP	Notification	US-13	Must Have
14	Implement email notifications	Notification	US-13, US-14	Must Have
15	Create the main worker script	Monitoring	US-15	Must Have
16	Configure environment variables/secrets	Security	US-18	Must Have
17	Create GitHub Actions workflow	Automation	US-16	Must Have

18	Configure scheduled execution	Automation	US-15, US-16	Must Have
19	Add error handling and logging	Reliability	US-19	Must Have
20	Add ESLint and Prettier	Code quality	US-20	Should Have
21	Add unit tests	Testing	US-20	Must Have
22	Add CI workflow	CI	US-20	Should Have
23	Test the complete workflow	Testing	US-20	Must Have
24	Document project setup and usage	Documentation	US-20	Must Have

10. Sprint Roadmap

The assignment deadline is November 1, 2026. The following four one-week Sprints provide a practical delivery plan. Meeting dates are planned dates; actual attendance, outcomes, and minutes must be recorded only after the real meetings occur.

Sprint	Dates	Sprint Goal	Main Backlog Work	Expected Increment
Sprint 1	Oct 10–11, 2026	Foundation and database	Node.js setup; Neon PostgreSQL; Sequelize; Coin model; Email model	Working project foundation and database models
Sprint 2	Oct 12–18, 2026	Administration and validation	Coin CRUD; Email CRUD; Zod validation; environment configuration	Administrator can manage coins/emails safely
Sprint 3	Oct 19–25, 2026	Monitoring and notification	CoinLore API; price retrieval; range checking; Gmail SMTP; notifications; worker	End-to-end monitoring and notification behavior
Sprint 4	Oct 26–Nov 1, 2026	Automation, testing and delivery	GitHub Actions; schedule; error handling; linting/formatting; tests; CI; complete test; documentation	Tested, automated, documented final increment

11. Sprint Goals and Backlogs

Sprint 1

Sprint Goal: Create the technical foundation and database structure.

- Set up Node.js project
- Set up Neon PostgreSQL database

- Configure Sequelize
- Create Coin model
- Create Email model

Sprint 2

Sprint Goal: Allow administrators to manage monitored coins and email recipients safely.

- Create Coin CRUD scripts
- Create Email CRUD scripts
- Add Zod validation for Coin scripts
- Add Zod validation for Email scripts
- Configure environment variables/secrets

Sprint 3

Sprint Goal: Implement the main cryptocurrency monitoring and notification functionality.

- Integrate CoinLore REST API
- Retrieve coin prices
- Implement price-range checking
- Integrate Gmail SMTP
- Implement email notifications
- Create main worker script

Sprint 4

Sprint Goal: Automate, test, document and prepare the complete system for final delivery.

- Create GitHub Actions workflow
- Configure scheduled execution
- Add error handling and logging
- Add ESLint and Prettier
- Add unit tests
- Add CI workflow
- Test complete workflow
- Document project setup and usage

12. Definition of Done

- The user story implementation is complete.
- All acceptance criteria are satisfied.
- The implementation has been tested.
- No known critical defect remains for the story.
- Code has been committed/pushed to GitHub according to the team's workflow.
- Required documentation has been updated.
- The Product Owner has reviewed the result.
- The Product Owner accepts the story during Sprint Review.

13. Sprint Planning Agenda

- Review the Product Vision and Sprint objective.

- Review the highest-priority Product Backlog items.
- Explain user stories and acceptance criteria.
- Developers estimate the work and identify dependencies.
- Select the Sprint Backlog based on team capacity.
- Agree on the Sprint Goal.
- Identify risks and blockers.
- Confirm the Definition of Done and Sprint schedule.

14. Sprint Review Agenda

- Review the Sprint Goal.
- Demonstrate the completed increment.
- Review each completed story against its acceptance criteria.
- Product Owner accepts or rejects completed stories.
- Record feedback and unfinished work.
- Update the Product Backlog and priorities.
- Confirm the focus for the next Sprint.

15. Product Owner Acceptance Record

User Story	Acceptance Criteria Met?	Status	Product Owner Decision	Notes
US-01	Yes / No	Complete / Incomplete	Accepted / Rejected	
US-02	Yes / No	Complete / Incomplete	Accepted / Rejected	
US-03	Yes / No	Complete / Incomplete	Accepted / Rejected	
US-04	Yes / No	Complete / Incomplete	Accepted / Rejected	
US-05	Yes / No	Complete / Incomplete	Accepted / Rejected	
US-06	Yes / No	Complete / Incomplete	Accepted / Rejected	
US-07	Yes / No	Complete / Incomplete	Accepted / Rejected	
US-08	Yes / No	Complete / Incomplete	Accepted / Rejected	
US-09	Yes / No	Complete / Incomplete	Accepted / Rejected	
US-10	Yes / No	Complete / Incomplete	Accepted / Rejected	
US-11	Yes / No	Complete / Incomplete	Accepted / Rejected	
US-12	Yes / No	Complete / Incomplete	Accepted / Rejected	
US-13	Yes / No	Complete / Incomplete	Accepted / Rejected	
US-14	Yes / No	Complete / Incomplete	Accepted / Rejected	
US-15	Yes / No	Complete / Incomplete	Accepted / Rejected	

US-16	Yes / No	Complete / Incomplete	Accepted / Rejected	
US-17	Yes / No	Complete / Incomplete	Accepted / Rejected	
US-18	Yes / No	Complete / Incomplete	Accepted / Rejected	
US-19	Yes / No	Complete / Incomplete	Accepted / Rejected	
US-20	Yes / No	Complete / Incomplete	Accepted / Rejected	

16. Daily Scrum Record Template

This template should be completed after each real Daily Scrum. Do not create fictional attendance, blockers, or outcomes.

Field	Record
Date	
Sprint	
Attendees	
What was completed yesterday?	
What will be worked on today?	
Blockers/impediments	
Product Owner requirement clarifications	
Backlog changes/decisions	

17. Sprint Review Minutes Template

Field	Record
Sprint	
Date	
Attendees	
Sprint Goal	
Completed stories	
Demonstrated features	
Accepted stories	
Rejected/incomplete stories	
Product Owner feedback	
New backlog items or priority changes	
Action items	

18. Product Owner Backlog Refinement Checklist

- The story has a clear user/business value.
- The story is understandable to the developers.
- Acceptance criteria are clear and testable.
- The story is small enough to be completed within a Sprint.
- Dependencies are identified.
- Priority is correct.
- The team can estimate the work.
- The story is ready for Sprint Planning.

19. Product Owner Risk Register

Risk	Probability	Impact	Product Owner Response
CoinLore API unavailable	Medium	High	Require API error handling and test failure behavior.
Gmail SMTP failure	Medium	High	Test email delivery early and require graceful error handling.
Neon database connection failure	Medium	High	Verify database connectivity early and document recovery/configuration.
GitHub Actions scheduling/configuration problem	Medium	High	Implement and test automation before the final Sprint.
Requirement misunderstanding	Medium	Medium	Clarify stories and acceptance criteria during Sprint Planning/Daily Scrum.
Sprint work exceeds capacity	Medium	Medium	Re-prioritize lower-value work and keep Must Have items first.
Sensitive credentials exposed	Low	High	Require GitHub Secrets/environment configuration and code review.
Insufficient end-to-end testing	Medium	High	Reserve Sprint 4 for complete workflow testing.

20. GitHub Documentation Checklist

- Product vision and objectives
- Requirements specification
- 20 user stories and acceptance criteria
- Product Backlog
- Sprint Backlogs
- Use Case and Sequence diagrams
- Sprint Planning agendas/minutes
- Daily Scrum agendas/minutes
- Sprint Review documentation
- Sprint Retrospective documentation
- Burndown charts
- Meeting recordings uploaded as required by the instructor
- README and setup/usage documentation

21. Product Owner Final Statement

As Product Owner, I am responsible for maximizing the value of the product by defining the product vision, maintaining and prioritizing the Product Backlog, clarifying requirements, writing and accepting user stories, participating in Sprint Planning and Sprint Reviews, and continuously adapting priorities based on progress

and feedback. The goal is to ensure that each Sprint produces a useful increment toward the automated cryptocurrency price monitoring system and that the final product satisfies the agreed requirements and acceptance criteria.

Prepared for the Software Engineering Scrum Project